portfolio-web — explications

Un site portfolio statique en HTML, CSS et jQuery : comment le refaire soi-même, étape par étape, et les mauvaises pratiques qu'on apprend à éviter en le codant.

Jean-Emmanuel Jacquier

Sommaire

1. Le projet en bref
2. Lancer et tester
3. Architecture
4. Refaire le projet soi-même, étape par étape
   - 4.1 Étape 1 : une page statique servie par HTTP
   - 4.2 Étape 2 : décrire les projets dans un JSON, et le valider
   - 4.3 Étape 3 : la logique de filtrage en fonctions pures
   - 4.4 Étape 4 : charger les données et afficher les cartes
   - 4.5 Étape 5 : la recherche et les filtres
   - 4.6 Étape 6 : la fiche détaillée (modale)
   - 4.7 Étape 7 : le thème clair ou sombre
   - 4.8 Étape 8 : animations et compteurs, sans imposer le mouvement
   - 4.9 Étape 9 : tests fiables, analyse statique et intégration continue
5. Les mauvaises pratiques qu'on apprend à éviter
   - 5.1 Attacher un écouteur à des éléments qui n'existent pas encore
   - 5.2 Lire les attributs `data-*` avec `.data()`
   - 5.3 Construire le HTML en concaténant des données
   - 5.4 Charger un script tiers sans vérifier son intégrité
   - 5.5 Supposer qu'une liste n'est jamais vide
   - 5.6 Avaler une erreur, ou l'afficher brute
   - 5.7 Laisser le focus au hasard
   - 5.8 Relancer une animation sans arrêter la précédente
   - 5.9 Animer en JavaScript sans respecter `prefers-reduced-motion`
   - 5.10 Un bouton bascule dont l'état n'est pas exposé
   - 5.11 Tester avec des pauses fixes, corriger sans test de non-régression
   - 5.12 Garder des valeurs par défaut dangereuses
6. Check-list avant de rendre son code
7. Pour aller plus loin
8. Glossaire

1. Le projet en bref

`portfolio-web` est le site personnel de Jean-Emmanuel Jacquier, étudiant en informatique à Epitech Lyon. C'est une seule page qui présente ses projets GitHub sous forme de cartes. Le visiteur peut rechercher un projet par mot-clé (sans se soucier des accents ni des majuscules), filtrer par catégorie, cliquer sur une compétence comme « Docker » pour ne voir que les projets qui l'utilisent, ouvrir la fiche détaillée d'un projet à la souris ou au clavier, et basculer entre thème clair et thème sombre.

Le site est **statique** : ni serveur applicatif ni base de données. Le navigateur télécharge `index.html`, une feuille de style, jQuery, deux scripts et le fichier `data/projets.json`, puis construit la page en JavaScript. Ajouter un projet consiste à ajouter une entrée au JSON, sans toucher au code.

Ce que le projet apprend à faire : séparer la logique (fonctions pures testables avec Node.js) de l'affichage (jQuery) ; manipuler le DOM sans ouvrir de faille d'injection ; gérer le clavier, le focus et les préférences du système (thème, animations réduites) ; écrire des tests de bout en bout fiables avec Playwright ; contrôler le code avec ESLint et Prettier ; et publier automatiquement le site seulement si tout passe.

Un exemple réel. La logique de filtrage ne dépend pas du navigateur : on peut l'appeler depuis Node.js pour savoir ce que la page affichera.

```
node -e '
const F = require("./js/filtres.js");
const p = require("./data/projets.json");
console.log(F.filtrerProjets(p, { recherche: "emissions" }).map(x => x.titre));
console.log(F.compterCompetences(p).slice(0, 2));'
```

```
# sortie réelle
[ 'Émissions de CO₂ en R' ]
[ { nom: 'Python', nombre: 5 }, { nom: 'Docker', nombre: 3 } ]
```

Dans la page, taper « emissions » laisse exactement cette carte visible, et « Python » est le mot le plus gros du nuage de compétences.

2. Lancer et tester

Le site doit être servi par HTTP (un double-clic sur `index.html` ne suffit pas, voir l'étape 4.1) :

```
npm start                  # python3 -m http.server 8080 --bind 127.0.0.1
                           # puis ouvrir http://127.0.0.1:8080
npm install
npm run lint               # ESLint (règles recommandées)
npm run format:check       # Prettier, sans rien modifier
npm test                   # 9 tests unitaires (node --test)
npx playwright install chromium
npm run test:e2e           # tests de bout en bout (Playwright)
```

`npm start` ne télécharge rien : il lance le serveur intégré à Python 3. Chaque scénario de `tests/portfolio.spec.js` est joué deux fois : sur un écran d'ordinateur et sur un téléphone simulé (Pixel 7). Le port 8080 doit être libre : Playwright lance son propre serveur et refuse d'en réutiliser un (5.12). Si un Chromium est déjà installé, la variable `CHROMIUM_PATH` permet de l'utiliser à la place de `npx playwright install` (voir `playwright.config.js`). À chaque `push`, `.github/workflows/ci.yml` relance ESLint, Prettier et tous les tests ; sur `main`, s'ils passent, il publie le site sur GitHub Pages (seulement si le dépôt est public : GitHub ne sert pas les dépôts privés d'un compte gratuit).

3. Architecture

```
push sur main --> GitHub Actions (ci.yml)
                    job tests : lint, format:check, npm test, npm run test:e2e
                    job deploiement (needs: tests) : index.html css js data --> Pages

Navigateur
  index.html     script inline du <head> : applique le thème mémorisé
  css/style.css  variables de couleur, thème sombre, prefers-reduced-motion
  jQuery (CDN)   vérifié par l'attribut integrity
  js/filtres.js  crée window.Filtres (fonctions pures, sans DOM)
  js/app.js      $(function () { ... }) quand le DOM est prêt
      |
      |  $.getJSON("data/projets.json")
      v
  projets[] --> afficherCategories()  afficherProjets()  afficherNuage()
                                            |
                 Filtres.filtrerProjets(projets, criteres)
                                            ^
       criteres = { recherche, categorie, competence }
                                            ^
  Événements : saisie #recherche, clic .filtre, clic .etiquette / .mot
               clic ou Entrée sur .carte --> ouvrirModale()
               Échap, fond, bouton Fermer --> fermerModale()
```

Le principe central : **un seul état** (`criteres`) et **une seule fonction de rendu** (`afficherProjets`). Chaque action de l'utilisateur modifie `criteres`, puis la grille est redessinée à partir de zéro.

Fichier	Rôle
`index.html`	Squelette : sections à remplir, modale cachée, script inline du thème
`css/style.css`	Variables de couleur, thème sombre, grille de cartes, modale, mobile
`js/filtres.js`	Fonctions pures : `normaliser`, `filtrerProjets`, `compterCompetences`, `categories`
`js/app.js`	Code jQuery : chargement, rendu, événements, modale, thème, compteurs
`data/projets.json`	Les 12 projets (titre, résumé, détails, compétences, catégorie, dépôt)
`tests/filtres.test.js`	Tests `node --test` des filtres et de la validité du JSON
`tests/portfolio.spec.js`	Tests Playwright : recherche, filtres, modale, clavier, thème, mobile
`playwright.config.js`	Serveur de test (`npm start`), profils « ordinateur » et « mobile »
`eslint.config.js`, `.prettierrc.json`	Analyse statique (ESLint) et mise en forme (Prettier)
`.github/workflows/ci.yml`	Lint et tests, puis déploiement GitHub Pages
`docs/`, `explications-pdf.yml`	Ce document, son script PDF et le workflow qui le régénère

4. Refaire le projet soi-même, étape par étape

Les étapes suivent l'ordre dans lequel on construirait réellement le site : d'abord la plus petite version qui fonctionne, puis une fonctionnalité à la fois, chacune vérifiée avant de passer à la suivante. Les extraits sont ceux du code final.

4.1 Étape 1 : une page statique servie par HTTP

Objectif. Écrire le squelette HTML (en-tête, sections vides, modale cachée) et l'ouvrir dans un navigateur comme il le sera en production. Les zones à remplir par JavaScript sont vides :

```
<div class="grille" id="grille"></div>
<p class="vide" hidden></p>
```

Vérifier. Lancer `npm start`, puis dans un autre terminal :

```
curl -s -o /dev/null -w '%{http_code} %{content_type}\n' \
  http://127.0.0.1:8080/data/projets.json
```

```
# sortie réelle (une fois data/projets.json créé à l'étape 2)
200 application/json
```

Le piège à cette étape. Ouvrir `index.html` par un double-clic (adresse `file://`). Dès l'étape 4, la page affiche « Impossible de charger les projets. » et la console de Chromium indique, en sortie réelle : `Access to XMLHttpRequest at 'file:///home/claude/portfolio-web/data/projets.json' from origin 'null' has been blocked by CORS policy`. Une page ouverte depuis le disque n'a pas d'origine et ne peut pas lire d'autres fichiers locaux. Autre piège : oublier `--bind` (5.12).

4.2 Étape 2 : décrire les projets dans un JSON, et le valider

Objectif. Séparer les données du code. Chaque projet de `data/projets.json` a un `id`, un `titre`, un `resume`, des `details`, une liste de `competences`, une `categorie` et un lien `depot` (exemple dans le `README.md`). Le premier test à écrire vérifie les données elles-mêmes (`tests/filtres.test.js`) :

```
test("chaque projet a les champs attendus", () => {
  const ids = new Set();
  for (const p of projets) {
    for (const champ of ["id", "titre", "resume", "details", "categorie", "depot"]) {
      assert.ok(p[champ], `${p.id} : champ ${champ} manquant`);
    }
    assert.ok(p.competences.length > 0, `${p.id} : aucune compétence`);
    assert.match(p.depot, /^https:\/\/github\.com\/jejcqr\//);
    assert.ok(!ids.has(p.id), `id en double : ${p.id}`);
    ids.add(p.id);
  }
});
```

Vérifier. `npm test` ; avec le projet complet, la sortie réelle se termine par `# tests 9`, `# pass 9`, `# fail 0`.

Le piège à cette étape. Une virgule après le dernier élément, tolérée en JavaScript, est interdite en JSON. Essai réel sur une copie du projet : `npm test` échoue dès le chargement, avec `SyntaxError: .../data/projets.json: Unexpected token ']', ... is not valid JSON`. C'est voulu : l'erreur est arrêtée avant la mise en ligne (5.12).

4.3 Étape 3 : la logique de filtrage en fonctions pures

Objectif. Écrire « quels projets afficher ? » sans jQuery ni `document`, dans un fichier que le navigateur **et** Node.js peuvent charger. Extrait de `js/filtres.js` :

```
(function (exports) {
  "use strict";

  function normaliser(texte) {
    return String(texte)
      .normalize("NFD")
      .replace(/[\u0300-\u036f]/g, "")
      .toLowerCase()
      .trim();
  }
  // ... filtrerProjets, compterCompetences, categories ...
  exports.normaliser = normaliser;
  exports.filtrerProjets = filtrerProjets;
  // ...
})(typeof module !== "undefined" ? module.exports : (window.Filtres = {}));
```

Le fichier réel écrit cette chaîne d'appels sur une seule ligne, et contient directement les deux accents combinants au lieu de `\u0300` et `\u036f` ; la forme échappée est plus lisible. `normalize("NFD")` sépare « é » en « e » suivi d'un accent combinant, que l'expression régulière supprime. `filtrerProjets` applique la catégorie, puis la compétence, puis exige que **chaque** mot de la recherche apparaisse dans le titre, le résumé, les détails ou les compétences. La dernière ligne range les fonctions dans `module.exports` sous Node.js, dans `window.Filtres` dans le navigateur ; `typeof` est la seule façon sûre de tester une variable qui peut ne pas exister (`module !== undefined` lèverait une `ReferenceError` dans le navigateur).

Vérifier. Écrire les tests en même temps que les fonctions, par exemple « la recherche ignore les accents et combine les mots », qui attend `["alhudabot"]` pour `{ recherche: "python discord coran" }`. L'appel `node -e` de la partie 1 donne aussi un retour immédiat.

Le piège à cette étape. Se contenter de `toLowerCase()` : en sortie réelle, `"Émissions".toLowerCase().includes("emissions")` vaut `false`. Et écrire le filtre dans un gestionnaire jQuery le rendrait intestable sans navigateur.

4.4 Étape 4 : charger les données et afficher les cartes

Objectif. Relier données et fonctions pures à la page. Fin de `js/app.js` :

```
$.getJSON("data/projets.json")
  .done(function (donnees) {
    projets = donnees;
    afficherCategories();
    afficherProjets();
    afficherNuage();
  })
  .fail(function (xhr, statut, erreur) {
    // Le détail technique va dans la console, pour le développeur ; le visiteur ne
    // voit qu'un message fixe, sans information interne.
    console.error("Chargement de data/projets.json impossible :", statut, String(erreur));
    $grille.html("<p class='vide'>Impossible de charger les projets.</p>");
  });
```

`$.getJSON` est **asynchrone** : `.done` est appelé plus tard, à l'arrivée de la réponse ; tout code qui a besoin des projets part de là. `afficherProjets` appelle `Filtres.filtrerProjets(projets, criteres)`, vide la grille avec `$grille.empty()`, puis crée chaque carte avec des balises fixes remplies par `.text()` et `.attr()` (5.3).

Vérifier. Premier test Playwright, « affiche tous les projets et les compteurs » : `await expect(page.locator(".carte")).toHaveCount(projets.length)`. L'attente vient des données : ajouter un projet ne casse pas le test.

Le piège à cette étape. Concaténer du HTML (5.3), oublier le `.fail` (5.6), supposer que le JSON contient au moins un projet (5.5).

4.5 Étape 5 : la recherche et les filtres

Objectif. Chaque action modifie `criteres` puis appelle `afficherProjets()`. Pour la recherche, on attend une pause dans la frappe (*debounce*) :

```
let minuterie;
$("#recherche").on("input", function () {
  const valeur = $(this).val();
  clearTimeout(minuterie);
  // Attendre une pause de 150 ms évite de tout redessiner à chaque touche.
  minuterie = setTimeout(function () {
    criteres.recherche = valeur;
    afficherProjets();
  }, 150);
});
```

La valeur est lue tout de suite, car dans la fonction différée `this` ne désigne plus le champ. `input` couvre aussi le collage et la croix d'effacement, contrairement à `keyup`. Les boutons de catégorie sont gérés par délégation : `$(".categories").on("click", ".filtre", ...)`.

Vérifier. Tests « la recherche filtre les cartes » et « les catégories filtrent les cartes ». Mesure réelle dans Chromium, en tapant « docker » à une touche toutes les 60 ms : 6 événements `input`, un seul appel à `Filtres.filtrerProjets`.

Le piège à cette étape. `$(".filtre").on("click", ...)` sur des boutons qui n'existent pas encore (5.1) ; `.data("categorie")`, qui convertit les types (5.2).

4.6 Étape 6 : la fiche détaillée (modale)

Objectif. Afficher le détail d'un projet par-dessus la page, utilisable à la souris, au clavier et avec un lecteur d'écran.

```
function ouvrirModale(id) {
  const p = projets.find(function (x) {
    return x.id === id;
  });
  if (!p) return;
  dernierFocus = document.activeElement;
  $modale.find("#modale-titre").text(p.titre);
  $modale.find(".modale-details").text(p.details);
  $modale.find(".modale-competences").empty().append(etiquettes(p.competences));
  $modale.find(".modale-lien").attr("href", p.depot);
  // stop(true, true) termine un fondu de fermeture en cours,
  // dont le rappel remettrait hidden.
  $modale.stop(true, true).prop("hidden", false).hide().fadeIn(150);
  // Ni clic, ni focus, ni lecteur d'écran derrière la modale.
  $arrierePlan.prop("inert", true);
  $modale.find(".modale-fermer").trigger("focus");
  $("body").addClass("sans-defilement");
}
```

La modale porte `role="dialog"`, `aria-modal="true"` et `aria-labelledby="modale-titre"` dans `index.html`. `fermerModale` fait l'inverse, retire `inert` **avant** de rendre le focus à `dernierFocus` (un élément inerte le refuse) et ne remet `hidden` qu'à la fin du fondu. Les cartes ont `tabindex='0'` et s'ouvrent aussi avec Entrée ou Espace.

Vérifier. Test « la modale s'ouvre au clic et se ferme avec Échap ».

Le piège à cette étape. Le CSS donne `display: grid` à `.modale`, ce qui l'emporte sur le style par défaut de `hidden` : sans `.modale[hidden] { display: none; }`, la modale est toujours visible. Pièges réels plus subtils : 5.7 et 5.8.

4.7 Étape 7 : le thème clair ou sombre

Objectif. Suivre la préférence du système, permettre de la remplacer, s'en souvenir. Dans le `<head>` de `index.html`, avant tout affichage :

```
<script>
  // Appliqué avant l'affichage pour éviter un flash de thème clair.
  try {
    const t = localStorage.getItem("theme");
    if (t) document.documentElement.dataset.theme = t;
  } catch {
    /* stockage indisponible : thème par défaut */
  }
</script>
```

Dans `css/style.css`, les couleurs sont des variables (`--fond`, `--texte`…) redéfinies sous `:root[data-theme="sombre"]` et sous `@media (prefers-color-scheme: dark)` avec `:root:not([data-theme="clair"])` (« sombre si le système l'est, sauf choix contraire »). Le reste du CSS n'utilise que `var(--fond)` : un seul attribut change tout.

Vérifier. Test « le thème sombre est mémorisé » : clic, `page.reload()`, puis `data-theme` vaut toujours `"sombre"`.

Le piège à cette étape. Appliquer le thème dans `app.js`, chargé en fin de page : l'affichage passerait du clair au sombre au chargement (illustratif, non mesuré ici). Le bouton doit aussi exposer son état (5.10).

4.8 Étape 8 : animations et compteurs, sans imposer le mouvement

Objectif. Ajouter les fondus en cascade et les compteurs animés, coupés pour les personnes qui ont demandé moins d'animations à leur système.

```
const moinsDeMouvement = window.matchMedia("(prefers-reduced-motion: reduce)");
$.fx.off = moinsDeMouvement.matches;
moinsDeMouvement.addEventListener("change", function (e) {
  $.fx.off = e.matches;
});
```

Pour les compteurs, `animerCompteur` appelle `$({ valeur: 0 }).animate({ valeur: cible }, { duration: 900, step: ..., complete: ... })` : `$()` enveloppe un simple objet dont `.animate` fait varier la propriété, `step` écrit la valeur arrondie et `complete` garantit le nombre final. Avec `$.fx.off` à `true`, jQuery saute directement à l'état final. Dans `afficherProjets`, le fondu n'est lancé que si les animations sont actives : `if (!$.fx.off) { $carte.hide().delay(i * 40).fadeIn(200); }` (sur plusieurs lignes dans le fichier).

Vérifier. Le test du bloc `avec prefers-reduced-motion` observe chaque valeur écrite dans le compteur et n'en attend qu'une, `"12"`.

Le piège à cette étape. Croire qu'une règle CSS `prefers-reduced-motion` suffit (5.9).

4.9 Étape 9 : tests fiables, analyse statique et intégration continue

Objectif. Rendre les tests reproductibles, contrôler le code automatiquement, puis ne publier que si tout passe.

```
test.beforeEach(async ({ page }) => {
  // jQuery est servi depuis node_modules : les tests ne dépendent pas du réseau.
  await page.route("https://code.jquery.com/**", (route) =>
    route.fulfill({
      path: path.join(__dirname, "../node_modules/jquery/dist/jquery.min.js"),
      contentType: "application/javascript",
    }),
  );
  await page.goto("/");
});
```

Cela ne marche que parce que `package.json` fixe `"jquery": "3.7.1"`, la version du CDN, sinon l'empreinte `integrity` ne correspondrait plus (5.4).

On ajoute ensuite ESLint et Prettier en dépendances de développement, avec deux scripts (`"lint": "eslint ."` et `"format:check": "prettier --check ."`). `eslint.config.js` (configuration « plate ») part de `js.configs.recommended` et déclare un environnement par type de fichier : navigateur et jQuery global (`globals.browser`, `globals.jquery`, `Filtres`) pour `js/`, avec la règle `eqeqeq` ; Node.js pour les tests et la configuration ; navigateur en plus pour les specs Playwright, dont les fonctions passées à `page.evaluate()` s'exécutent dans la page. `.prettierrc.json` fixe seulement `"printWidth": 100`.

Dans `.github/workflows/ci.yml`, le job `tests` lance `npm run lint && npm run format:check` avant les tests, et le job `deploiement` déclare `needs: tests`.

Vérifier. `npm run lint` et `npm run format:check` ne signalent rien (sortie réelle de Prettier : `All matched files use Prettier code style!`). `CHROMIUM_PATH=/opt/pw-browsers/chromium-1194/chrome-linux/chrome npx playwright test --repeat-each=2` (chemin propre à la machine utilisée) se termine, en sortie réelle, par `72 passed (26.1s)` : les 36 tests, joués deux fois pour débusquer un test instable.

Le piège à cette étape. Attendre une durée au lieu d'un état, corriger sans test de non-régression (5.11) ; publier sans `needs: tests`, réutiliser un serveur déjà lancé ou donner trop de permissions à la CI (5.12). À sa première exécution, ESLint a signalé `'e' is defined but never used` sur le `catch (e)` du thème (5.6).

5. Les mauvaises pratiques qu'on apprend à éviter

Chaque sous-partie part du code qu'un débutant écrirait spontanément dans ce projet. La mention « Piège réel rencontré dans ce projet » signale un défaut d'une version précédente du dépôt : soit la première version, corrigée lors de la relecture, soit la version de relecture elle-même, corrigée ensuite (journalisation des erreurs en 5.6, permissions de la CI et serveur de test en 5.12). Chaque correction est accompagnée d'un test quand le défaut est testable, et la conséquence décrite a été reproduite en lançant l'ancien code. Les autres conséquences ont été reproduites dans la page actuelle, sauf mention « illustratif ».

5.1 Attacher un écouteur à des éléments qui n'existent pas encore

Piège réel rencontré dans ce projet (deuxième forme).

Ce qu'on écrit spontanément. `$(".carte").on("click", ...)` ; ou, une fois la délégation découverte, un gestionnaire clavier trop large (première version) :

```
$grille.on("keydown", ".carte", function (e) {
  if (e.key === "Enter" || e.key === " ") {
    e.preventDefault();
    ouvrirModale($(this).data("id"));
  }
});
```

Ce qui se passe. `$(".carte")` s'exécute avant la réponse de `$.getJSON` : il ne trouve aucun élément et le gestionnaire n'est attaché à rien, sans erreur. Essai réel avec jQuery 3.7.1 : après `$(".x").on("click", () => n++)` et `$("#g").on("click", ".x", () => n += 10)`, puis l'ajout d'un `.x` dans `#g` et un clic, `n` vaut 10, pas 11. Pour le second extrait, l'appui sur Entrée dans l'étiquette « Docker » d'une carte remonte jusqu'à la carte : la fiche s'ouvre et `preventDefault()` annule le clic que le navigateur aurait déclenché sur le bouton. Mesure réelle sur la première version : modale ouverte, aucun filtre, 12 cartes au lieu de 3.

Pourquoi c'est une mauvaise pratique. Un sélecteur est évalué une seule fois : il ignore les éléments créés ensuite, et ceux qu'il a trouvés disparaissent au premier `empty()`. Inversement, un événement se propage vers tous les ancêtres (*bubbling*) : un gestionnaire posé sur un conteneur reçoit aussi les événements de ses descendants, et son `preventDefault()` peut casser un comportement natif plus bas.

La bonne pratique. Déléguer à un parent stable, et vérifier la cible avant d'agir :

```
$grille.on("keydown", ".carte", function (e) {
  // Seulement sur la carte elle-même : Entrée sur une étiquette doit garder son clic.
  if (e.target !== this) return;
  if (e.key === "Enter" || e.key === " ") {
    e.preventDefault();
    ouvrirModale($(this).attr("data-id"));
  }
});
```

Les étiquettes (cartes, modale, nuage) sont déléguées à `document`, et leur gestionnaire appelle `e.stopPropagation()` pour qu'un clic sur une étiquette n'ouvre pas la fiche.

Comment s'en protéger durablement. Le test « Entrée sur une étiquette d'une carte filtre sans ouvrir la fiche ». Réflexes : tester chaque interaction au clavier, pas seulement à la souris ; un gestionnaire qui appelle `preventDefault()` vérifie d'abord `e.target`.

> **À retenir :** pour des éléments créés dynamiquement, on délègue à un conteneur stable avec `.on("evenement", "selecteur", ...)`, et on compare `e.target` à `this` avant d'annuler quoi que ce soit.

5.2 Lire les attributs `data-*` avec `.data()`

Piège réel rencontré dans ce projet.

Ce qu'on écrit spontanément (première version) :

```
$grille.on("click", ".carte", function () { ouvrirModale($(this).data("id")); });
// dans ouvrirModale :
var p = projets.find(function (x) { return x.id === id; });
if (!p) return;
```

Ce qui se passe. `.data()` convertit une valeur qui ressemble à un nombre, un booléen ou du JSON. Essai réel : `$("<div data-id='42'>").data("id")` renvoie le nombre `42`, `.attr("data-id")` la chaîne `"42"`. Avec un projet d'identifiant `"42"`, `"42" === 42` est faux, `find` ne trouve rien et `return` fait échouer l'ouverture en silence. Sur la première version, le test correspondant échoue avec `Expected: "Projet 42"`, `Received: ""`.

Pourquoi c'est une mauvaise pratique. Une conversion implicite rend le comportement dépendant du **contenu** des données : le code marche avec `"go-uptime"`, casse avec `"42"`, et aucun test écrit avec les données actuelles ne le voit.

La bonne pratique. `.attr()`, qui renvoie toujours la chaîne, avec un commentaire pour le prochain lecteur. Idem pour `data-categorie` et `data-competence`.

```
// .attr() et non .data(), qui convertirait data-id="42" en nombre.
$grille.on("click", ".carte", function () {
  ouvrirModale($(this).attr("data-id"));
});
```

Comment s'en protéger durablement. Le test « un identifiant numérique ouvre quand même la bonne fiche » sert, avec `page.route`, un projet d'identifiant `"42"` et de compétence `"2024"`. Réflexes : tester avec des données limites fabriquées ; `===` partout.

> **À retenir :** `.data()` convertit les types ; pour lire une chaîne, `.attr("data-…")`.

5.3 Construire le HTML en concaténant des données

Ce qu'on écrit spontanément.

```
$grille.append("<article class='carte'><h3>" + p.titre + "</h3>"
  + "<p>" + p.resume + "</p></article>");
```

Ce qui se passe. Essai réel dans la page, avec le titre `<img src=x onerror="window.pirate = 'code exécuté'">` : inséré par concaténation, `window.pirate` vaut `'code exécuté'` 300 ms plus tard (l'image échoue à charger et `onerror` exécute le code) ; inséré avec `$("<h3>").text(titre)`, aucune image n'est créée et le titre s'affiche tel quel. C'est une faille **XSS** : le code injecté pourrait modifier les liens vers les dépôts ou agir au nom du visiteur. Même sans malveillance, un résumé contenant « la balise `<main>` », inséré comme HTML, perdrait le mot `<main>`.

Pourquoi c'est une mauvaise pratique. Mélanger données et code dans une même chaîne laisse les données décider de la structure : c'est le même défaut que l'injection SQL. Une donnée est non fiable par principe, même si elle vient aujourd'hui d'un fichier écrit par l'auteur.

La bonne pratique. Balises fixes, données placées par `.text()` ou `.attr()` :

```
.append($("<h3>").text(p.titre))
.append($("<p>").text(p.resume))
```

Le seul `.html()` de `js/app.js` reçoit une chaîne fixe, dans le `.fail`.

Comment s'en protéger durablement. Relire le résultat de `grep -nE '\.(html|append)\(' js/app.js` : chaque appel doit recevoir un objet jQuery ou une chaîne fixe, jamais une chaîne construite avec des données. Les règles recommandées d'ESLint ne détectent pas ce défaut ; l'exercice 7.5 propose une règle qui le ferait.

> **À retenir :** une donnée ne doit jamais être insérée comme du HTML. Structure fixe, puis `.text()` ou `.attr()`.

5.4 Charger un script tiers sans vérifier son intégrité

Ce qu'on écrit spontanément. `<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>`

Ce qui se passe. Le navigateur exécuterait tout ce que le CDN renvoie. Avec l'attribut `integrity` du projet, un essai réel où l'on sert un jQuery auquel on a seulement ajouté un commentaire `/* modifié */` donne dans la console :

```
Failed to find a valid digest in the 'integrity' attribute for resource
'https://code.jquery.com/jquery-3.7.1.min.js' with computed SHA-256 integrity
'f5kHpRup4dTh/ORBmpobIEW5UT/0dM44oVQIraYbhmU='. The resource has been blocked.
```

Le script est refusé et `app.js` s'arrête sur `$ is not defined` : aucune carte, mais aucun code étranger exécuté. Sans l'attribut, le fichier modifié aurait été exécuté.

Pourquoi c'est une mauvaise pratique. Tout code chargé depuis un tiers fait partie de l'application ; sans vérification, la sécurité du site dépend de celle du CDN. C'est le principe de `npm ci` et de `package-lock.json` : exécuter exactement ce qu'on a vérifié.

La bonne pratique.

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"
        integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs0OEvtFFmqPF/lYI/Cxo="
        crossorigin="anonymous"></script>
```

Dans le même esprit, le lien GitHub de la modale (`target="_blank"`) porte `rel="noopener"` : la page ouverte ne peut pas manipuler l'onglet du portfolio.

Comment s'en protéger durablement. Les tests Playwright servent le jQuery de `node_modules` (version 3.7.1 exacte) à la place du CDN : si l'empreinte ne correspond pas, la page ne fonctionne plus et tous les tests échouent. Mettre jQuery à jour oblige à changer `package.json` et `integrity` ensemble.

> **À retenir :** un script venu d'un CDN porte toujours `integrity`, et la version est fixe partout (`3.7.1`, pas `^3`).

5.5 Supposer qu'une liste n'est jamais vide

Piège réel rencontré dans ce projet.

Ce qu'on écrit spontanément (première version, dans `afficherNuage`) :

```
var stats = Filtres.compterCompetences(projets);
var max = stats[0].nombre;
```

Ce qui se passe. Avec un `data/projets.json` réduit à `[]`, la première version lève, en sortie réelle, `Cannot read properties of undefined (reading 'nombre')`. L'exception interrompt `afficherNuage` ; la page affiche « Aucun projet ne correspond à cette recherche. » alors qu'aucune recherche n'a été faite, et garde « Cliquez sur une compétence… » au-dessus d'un nuage vide.

Pourquoi c'est une mauvaise pratique. `tableau[0]` sur un tableau vide renvoie `undefined` sans erreur ; l'erreur apparaît plus loin, loin de sa cause. Les cas limites (vide, un seul élément) sont ceux que les données réelles ne montrent jamais pendant le développement.

La bonne pratique.

```
// Liste triée : la première compétence est la plus utilisée.
const max = stats.length ? stats[0].nombre : 0;
if (!stats.length) $(".aide").text("Aucune compétence à afficher pour le moment.");
```

`afficherProjets` distingue de même « Aucun projet à afficher pour le moment. » (JSON vide) de « Aucun projet ne correspond à cette recherche. ».

Comment s'en protéger durablement. Le test unitaire « une liste de projets vide donne des résultats vides » et le test Playwright « une liste de projets vide affiche un message au lieu de planter », qui collecte les `pageerror` et exige qu'il n'y en ait aucun. Ce réflexe se généralise : un test de bout en bout devrait échouer sur toute exception JavaScript.

> **À retenir :** avant d'écrire `liste[0]`, se demander « et si la liste est vide ? », et le tester avec des données fabriquées.

5.6 Avaler une erreur, ou l'afficher brute

Piège réel rencontré dans ce projet (erreur avalée), corrigé avec un test écrit d'abord.

Ce qu'on écrit spontanément. Une requête sans `.fail`, ou un `.fail` qui affiche un message mais jette la cause, comme la version précédente du projet :

```
.fail(function () {
  $grille.html("<p class='vide'>Impossible de charger les projets.</p>");
});
```

Ou, à l'inverse, montrer l'erreur au visiteur (illustratif) : `.fail(function (xhr, statut, erreur) { $grille.text(statut + " : " + erreur); })`.

Ce qui se passe. Essai réel avec `data/projets.json` remplacé par le texte `[ { invalide` : sans `.fail`, la grille reste vide et le compteur affiche « 0 » ; avec l'ancien `.fail`, le visiteur lit « Impossible de charger les projets. ». Dans les deux cas, **la console ne contient aucun message** : le développeur n'a aucune piste. Le test écrit avant la correction l'a confirmé en échouant sur les deux profils (`Expected substring: "parsererror"`, `Received string: ""`). Dans la version « erreur brute », le visiteur lirait `parsererror : SyntaxError: Expected property name or '}' in JSON at position 4 (line 1 column 5)` (arguments réels du `.fail`), incompréhensible et révélateur de détails internes.

Pourquoi c'est une mauvaise pratique. Une opération réseau peut toujours échouer. Une erreur avalée transforme une panne en mystère ; une erreur brute donne à l'utilisateur une information inutile et, dans une vraie application, peut révéler chemins, versions ou requêtes. La règle : message clair pour l'utilisateur, détail pour le développeur.

La bonne pratique. Le `.fail` actuel (extrait en 4.4) sépare les deux publics : `console.error("Chargement de data/projets.json impossible :", statut, String(erreur))` pour le développeur, un message fixe pour le visiteur. Sortie réelle de la console avec le même JSON invalide :

```
Chargement de data/projets.json impossible : parsererror SyntaxError: Expected property name or '}' in JSON at position 4 (line 1 column 5)
```

Pour `localStorage`, dont l'accès peut lever une exception quand le stockage est bloqué, l'erreur est en revanche ignorée **volontairement** (le thème non mémorisé n'empêche pas d'utiliser le site), et le code le dit : `catch { /* stockage indisponible */ }`. ESLint signalait l'ancienne forme `catch (e)` (`'e' is defined but never used`) ; la forme sans variable, valide depuis ES2019, exprime mieux l'intention.

Comment s'en protéger durablement. Le test « un JSON invalide affiche un message clair et journalise la cause » sert le JSON invalide avec `page.route`, vérifie le message

affiché, l'absence de `SyntaxError` dans la page, et la présence de `parsererror` dans la console (`page.on("console", ...)`). Réflexes : chaque requête a sa branche d'échec, qui journalise ; chaque `catch` vide porte un commentaire qui justifie le silence.

> **À retenir :** jamais d'erreur avalée en silence ni affichée brute : message clair pour l'utilisateur, `console.error` pour le développeur. Un `catch` vide n'est acceptable que justifié par écrit.

5.7 Laisser le focus au hasard

Deux pièges réels rencontrés dans ce projet.

Ce qu'on écrit spontanément (première version) : une modale qui met le focus sur « Fermer » à l'ouverture, sans plus ; un filtre par compétence qui ferme la modale, redessine la grille et fait défiler la page, sans s'occuper du focus.

Ce qui se passe. Mesures réelles sur la première version, fiche « mysh » ouverte au clavier :

- Tab parcourt « C », « Linux », le lien GitHub, puis sort de la modale vers `<body>`, puis vers le logo de l'en-tête, **derrière** le fond sombre ;
- clic sur l'étiquette « Linux » de la modale : `fermerModale()` rend le focus à la carte d'origine, qu'`afficherProjets()` détruit aussitôt ; le focus retombe sur `<body>`, au début de la page, et un lecteur d'écran n'annonce rien du filtre.

Pourquoi c'est une mauvaise pratique. Pour qui navigue au clavier ou avec un lecteur d'écran, le focus **est** la position dans la page. `aria-modal="true"` promet que rien d'autre n'est utilisable : le code doit tenir cette promesse.

La bonne pratique. L'arrière-plan devient `inert` (4.6), et Tab boucle dans la modale :

```
$modale.on("keydown", function (e) {
  if (e.key !== "Tab") return;
  const $elements = $modale.find("a[href], button:not(:disabled)");
  const premier = $elements.get(0),
    dernier = $elements.get(-1);
  if (e.shiftKey && document.activeElement === premier) {
    e.preventDefault();
    dernier.focus();
  } else if (!e.shiftKey && document.activeElement === dernier) {
    e.preventDefault();
    premier.focus();
  }
});
```

`filtrerParCompetence` place lui-même le focus sur le bandeau « Filtré par … », qui porte `tabindex="-1"` (focalisable par le code, pas par Tab) : `$(".filtre-actif")[0].focus({ preventScroll: true })`. « tout afficher », qui cache ce bandeau, renvoie le focus au champ de recherche. Mesure réelle sur la version actuelle : après le lien GitHub,

Tab revient sur « Fermer » ; après « Linux », le focus est sur le bandeau et le Tab suivant atteint « tout afficher ».

Comment s'en protéger durablement. Les tests « le focus reste piégé dans la modale et l'arrière-plan est inerte » et « après un filtre lancé depuis la modale, le focus va sur le bandeau du filtre » (`toBeFocused()`). Réflexe : chaque fois que le code cache ou détruit un élément (`empty()`, `hidden`, `fadeOut`), se demander s'il pouvait avoir le focus, et où celui-ci doit aller.

> **À retenir :** le focus est un état que le code gère explicitement. Une modale le garde et rend l'arrière-plan inerte ; avant de détruire l'élément focalisé, on décide où mettre le focus.

5.8 Relancer une animation sans arrêter la précédente

Piège réel rencontré dans ce projet.

Ce qu'on écrit spontanément (première version) :

```
$modale.prop("hidden", false).hide().fadeIn(150);                    // ouvrirModale
$modale.fadeOut(150, function () { $modale.prop("hidden", true); });  // fermerModale
```

Ce qui se passe. Échap puis, tout de suite, Entrée sur la carte : la fiche se rouvre **pendant** le fondu de fermeture, dont le rappel remet ensuite `hidden`. Mesure réelle sur la première version, 600 ms plus tard : attribut `hidden` présent, mais `display: block` (style en ligne posé par jQuery) et opacité 1. La fiche est affichée mais marquée cachée, et Échap ne fait plus rien, puisque `fermerModale` commence par `if ($modale.prop("hidden")) return;`.

Pourquoi c'est une mauvaise pratique. Une animation est une opération asynchrone à rappel différé : c'est une situation de concurrence, comme deux requêtes dont les réponses arrivent dans le désordre. Un rappel ne doit pas supposer que l'état n'a pas changé depuis son lancement.

La bonne pratique. `stop(true, true)` vide la file d'animations et termine celle en cours, rappel compris, **avant** la réouverture :

```
$modale.stop(true, true).prop("hidden", false).hide().fadeIn(150);
```

Comment s'en protéger durablement. Le test « fermer puis rouvrir aussitôt la fiche la laisse utilisable » enchaîne `Escape` et `Enter` sans attente. Réflexe : pour toute action animée, tester l'action inverse déclenchée pendant l'animation.

> **À retenir :** un rappel différé (animation, minuteur, requête) s'exécute dans un état qui a pu changer ; annuler l'opération précédente (`stop`, `clearTimeout`) d'abord.

5.9 Animer en JavaScript sans respecter `prefers-reduced-motion`

Deux pièges réels rencontrés dans ce projet.

Ce qu'on écrit spontanément (première version) : une règle CSS qui coupe les transitions sous `prefers-reduced-motion`, des fondus et compteurs jQuery, et un défilement doux programmé pour les liens du menu :

```
$(".nav a[href^='#']").on("click", function (e) {
  var $cible = $($(this).attr("href"));
  if ($cible.length) {
    e.preventDefault();
    $("html, body").animate({ scrollTop: $cible.offset().top - 70 }, 400);
  }
});
```

Ce qui se passe. Mesures réelles sur la première version avec `reducedMotion: "reduce"` : le compteur « projets » reçoit 57 valeurs successives au lieu d'une, car la règle CSS ne voit pas les `.animate()` jQuery ; après Entrée sur « Projets », l'adresse reste sans `#projets` et le Tab suivant va sur le lien « Compétences » du menu au lieu du champ de recherche, car `preventDefault()` a empêché le navigateur de suivre l'ancre.

Pourquoi c'est une mauvaise pratique. Une préférence de l'utilisateur (troubles vestibulaires, migraines) se respecte partout, pas seulement là où c'est facile. Et réécrire un comportement natif fait perdre ce que le navigateur faisait gratuitement : adresse, historique, point de départ du clavier.

La bonne pratique. `$.fx.off` réglé d'après la même requête média (4.8) ; `.delay()`, simple minuteur qui ignore `$.fx.off`, n'est utilisé que si les animations sont actives. Le défilement doux passe au CSS, et les liens du menu n'ont plus de JavaScript :

```
html {
  scroll-behavior: smooth;
  scroll-padding-top: 70px; /* ancres sous l'en-tête collant */
}
@media (prefers-reduced-motion: reduce) {
  * {
    transition: none !important;
    scroll-behavior: auto !important;
  }
}
```

Mesure réelle sur la version actuelle : adresse en `#projets`, Tab sur le champ de recherche.

Comment s'en protéger durablement. `test.use({ reducedMotion: "reduce" })` et le test « après un lien du menu, Tab continue dans la section visée ». Réflexe : avant un `preventDefault()` sur un lien, se demander ce que le navigateur aurait fait.

> **À retenir :** `prefers-reduced-motion` coupe toutes les animations, y compris en JavaScript (`$.fx.off`, mais pas `.delay()`). Préférer le comportement natif à sa réimplémentation.

5.10 Un bouton bascule dont l'état n'est pas exposé

Piège réel rencontré dans ce projet.

Ce qu'on écrit spontanément (première version) : un bouton `<button class="bouton-theme" type="button" aria-label="Changer de thème">☾</button>`, et la logique « le thème est-il sombre ? » écrite directement dans le gestionnaire de clic.

Ce qui se passe. Mesure réelle sur la première version, système en thème sombre : nom « Changer de thème », `aria-pressed` absent, icône « ☾ » alors que la page est déjà sombre. Un lecteur d'écran annonce « Changer de thème, bouton » sans dire quel thème est actif, et l'icône ne change jamais.

Pourquoi c'est une mauvaise pratique. Un état visible doit être lisible par les technologies d'assistance, et calculé à **un seul** endroit : enfermé dans le gestionnaire de clic, il ne pouvait servir à rien d'autre.

La bonne pratique. Nom fixe (`aria-label="Thème sombre"`), état dans `aria-pressed`, une fonction d'état appelée au chargement, au clic et au changement de préférence du système :

```
function themeSombre() {
  const choix = $("html").attr("data-theme");
  return choix ? choix === "sombre" : systemeSombre.matches;
}
function majBoutonTheme() {
  $boutonTheme.attr("aria-pressed", themeSombre());
}
```

Le nom ne change pas avec l'état (« Passer au thème clair » annoncé « activé » serait contradictoire). L'icône, décorative, vient du CSS : `.bouton-theme::before { content: var(--icone-theme); }`. Mesure réelle sur la version actuelle, système sombre : « Thème sombre », `aria-pressed="true"`, icône « ☀ ».

Comment s'en protéger durablement. `toHaveAccessibleName` et `toHaveAttribute("aria-pressed", …)` dans deux tests, dont un qui change la préférence avec `page.emulateMedia`. Réflexe : inspecter l'arbre d'accessibilité de chaque contrôle personnalisé.

> **À retenir :** un bouton bascule a un nom fixe et un état `aria-pressed`, calculé par une seule fonction appelée partout où il peut changer.

5.11 Tester avec des pauses fixes, corriger sans test de non-régression

Ce qu'on écrit spontanément (illustratif) : `await page.waitForTimeout(1000)` avant `expect(await page.locator(".carte").count()).toBe(2)` ; et, pour corriger un bug, modifier `app.js`, vérifier à la main, commiter.

Ce qui se passe. Le test à pause fixe est lent quand tout va bien et faux quand la machine de CI est chargée (illustratif). La correction sans test ne protège de rien : le bug peut revenir au prochain remaniement sans que personne ne le voie.

Démarche réelle dans ce projet : les 17 scénarios de la suite écrite à la relecture ont été rejoués contre la première version, extraite à part avec `git archive <commit> | tar -x -C <dossier>` (port du serveur de test changé dans la copie pour ne pas réutiliser celui de la version actuelle, voir 5.12). Sortie réelle : `20 failed`, `14 passed (1.1m)`. Les dix scénarios ajoutés par ce commit échouent, chacun sur le défaut qu'il vise, sur les deux profils ; sur la version corrigée, les 34 tests d'alors passent. Même démarche pour la journalisation des erreurs (5.6) : le nouveau test a d'abord échoué (2 échecs), puis réussi après la correction. Un test qui n'a jamais échoué ne prouve rien.

Pourquoi c'est une mauvaise pratique. Un test vérifie un **comportement observable** et attend un **état**, jamais une durée. Écrit **avant** la correction, un test de non-régression prouve que le bug est reproduit ; le voir passer ensuite prouve que la correction le corrige.

La bonne pratique. Les assertions Playwright réessaient jusqu'à être vraies, ou jusqu'à un délai maximal :

```
await page.click(".filtre[data-categorie='bots']");
await expect(page.locator(".carte")).toHaveCount(2);
```

Comment s'en protéger durablement. Pour chaque bug : le test qui échoue, puis la correction, puis les deux dans le même commit. `grep -rn waitForTimeout tests/` doit ne rien afficher. La CI rejoue tout à chaque `push`.

> **À retenir :** attendre un état, pas une durée. Pour chaque bug corrigé, un test qui échouait avant et passe après.

5.12 Garder des valeurs par défaut dangereuses

Deux pièges réels rencontrés dans ce projet : permissions de la CI, serveur de test réutilisé.

Ce qu'on écrit spontanément. `python3 -m http.server 8080` ; des `permissions` déclarées une fois pour tout le workflow, comme dans la version précédente de `ci.yml` ; et, dans `playwright.config.js`, l'option `reuseExistingServer: true` que propose la documentation

de Playwright. L'ancien `ci.yml` déclarait en tête `permissions: { contents: read, pages: write, id-token: write }` (forme abrégée).

Ce qui se passe. Sortie réelle du serveur Python lancé sans `--bind` : `Serving HTTP on 0.0.0.0 port 8097 (http://0.0.0.0:8097/) ...`. `0.0.0.0` signifie « toutes les interfaces » : sur le Wi-Fi d'une école, tout appareil du réseau peut lire le dossier servi. Les permissions de niveau workflow sont accordées à **chaque** job : le job `tests`, qui exécute `npm ci` et donc le code de dizaines de paquets, recevait le droit de publier sur Pages et d'obtenir un jeton OIDC. Enfin, essai réel : un autre site (une page « autre ») est lancé sur le port 8080, puis `npx playwright test -g "affiche tous les projets"`. Playwright réutilise ce serveur sans prévenir et teste le mauvais site (`Received: 0` cartes). Avec une ancienne version du portfolio, ils auraient pu **passer** sur un autre code que le code courant.

Pourquoi c'est une mauvaise pratique. Une valeur par défaut vise à ce que « ça marche », pas à être sûre dans votre cas. **Moindre privilège** : n'écouter, ne publier et n'autoriser que le nécessaire ; et tester le code qu'on croit tester.

La bonne pratique. `"start": "python3 -m http.server 8080 --bind 127.0.0.1"` dans `package.json`. Dans `ci.yml`, un minimum par défaut et des permissions par job ; le déploiement attend les tests et ne publie qu'un dossier préparé (`_site`) :

```yaml
# Minimum par défaut ; chaque job déclare exactement ce dont il a besoin.
permissions:
  contents: read

jobs:
  # tests : permissions: { contents: read }
  deploiement:
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    needs: tests
    permissions:
      contents: read
      pages: write # publier sur GitHub Pages
      id-token: write # jeton OIDC temporaire exigé par deploy-pages
```

(Extrait abrégé.) `explications-pdf.yml` suit la même règle (`contents: write` pour le seul job qui commite le PDF). Dans `playwright.config.js`, `reuseExistingServer: false`. La variante `!process.env.CI` (réutiliser en local, jamais en CI) n'a pas été retenue : c'est justement en local que traîne un vieux serveur. Avec le même autre site sur le port 8080, Playwright s'arrête désormais avant tout test, en sortie réelle :

```
Error: http://127.0.0.1:8080 is already used, make sure that nothing is running on the
port/url or set reuseExistingServer:true in config.webServer.
```

La contrepartie est voulue : il faut arrêter `npm start` avant `npm run test:e2e`. Aucun secret n'est stocké ; l'adresse e-mail de `index.html` est publique par choix.

Comment s'en protéger durablement. Lire la ligne de démarrage d'un serveur ; pour chaque option, se demander « que se passe-t-il sans elle ? » ; relire les `permissions` de chaque job (elles ne se testent pas en local) ; protéger la branche `main`.

> **À retenir :** ne jamais se fier aux valeurs par défaut pour l'exposition réseau, les permissions ou la réutilisation d'un serveur. Moindre privilège, job par job, et déploiement conditionné aux tests.

6. Check-list avant de rendre son code

Mauvaise pratique	Risque	Bonne pratique	Où dans le projet
Écouteur sur éléments absents ; `preventDefault()` délégué	Action sans effet, clic annulé	Délégation, `e.target !== this`	gestionnaires `.carte`
`.data()` pour lire `data-*`	`"42"` devient `42`	`.attr()`, `===`	`.attr("data-id")`
HTML concaténé avec des données	XSS	Balises fixes + `.text()`	`afficherProjets`
CDN sans `integrity`	Code modifié exécuté	SRI, version fixe	`index.html`
`liste[0]` sans tester le vide	`TypeError`	Tester `length`	`afficherNuage`
Erreur avalée ou affichée brute	Panne sans piste, fuite d'infos	Message fixe + `console.error`	`.fail`, thème
Focus non géré	Focus perdu ou caché	`inert`, boucle Tab, focus placé	`ouvrirModale`, `filtrerParCompetence`
Animation non arrêtée	Modale impossible à fermer	`stop(true, true)`	`ouvrirModale`
Animations JS, ancres en JS	Mouvement imposé, focus perdu	`$.fx.off`, CSS	`js/app.js`, `css/style.css`
Bascule sans état	État invisible	`aria-pressed`, nom fixe	`majBoutonTheme`
`waitForTimeout`, correctif sans test	Tests instables, régressions	Attendre un état ; test d'abord	`tests/portfolio.spec.js`
Défauts dangereux	Dossier exposé, droits en trop, mauvais serveur testé	`--bind`, permissions par job, `reuseExistingServer: false`	`package.json`, `ci.yml`, `playwright.config.js`
Code non vérifié automatiquement	Variables inutilisées, `==`, style incohérent	ESLint + Prettier, en CI	`eslint.config.js`, `package.json`
Variables globales	Collisions entre scripts	Fonction englobante, `"use strict"`	`app.js`, `filtres.js`
`!== undefined` sur une variable absente	`ReferenceError`	`typeof x !== "undefined"`	fin de `filtres.js`
`toLowerCase()` seul	Accents non ignorés	`normalize("NFD")`, `localeCompare`	`normaliser`

Commandes à lancer avant chaque commit, depuis la racine du dépôt (les quatre premières sont aussi celles de la CI) :

```
npm run lint                                  # ESLint : aucune sortie = OK
npm run format:check                          # Prettier ; corriger avec npm run format
npm test                                      # 9 tests unitaires, dont la validité du JSON
npm run test:e2e                              # tests Playwright (port 8080 libre)
grep -nE '\.(html|append)\(' js/app.js        # relire : pas de HTML construit
git diff --check                              # espaces en fin de ligne
bash docs/generer-pdf.sh                      # si la documentation a changé
```

Après une correction délicate, `npx playwright test --repeat-each=2` rejoue chaque test deux fois pour détecter un test instable.

7. Pour aller plus loin

1. **Annoncer le nombre de résultats.** Pendant la frappe, un lecteur d'écran ne sait pas combien de cartes restent. Piste : un paragraphe `aria-live="polite"` que `afficherProjets()` remplit avec « 3 projets », et un test Playwright qui lit son texte.
2. **Vérifier l'accessibilité automatiquement.** Les tests actuels couvrent les défauts déjà rencontrés, pas les autres (contraste insuffisant, nom manquant). Piste : ajouter `@axe-core/playwright` et un test qui analyse la page en thème clair et sombre, modale ouverte et fermée, et exige zéro violation.
3. **Faire échouer tous les tests sur une erreur de console.** Aujourd'hui, seuls quelques tests surveillent `pageerror` ou `console`. Piste : une *fixture* Playwright automatique (`test.extend` avec `{ auto: true }`) qui collecte `pageerror` et les messages `console.error` inattendus, et fait échouer le test s'il y en a ; le test du JSON invalide déclare les siens comme attendus.
4. **Épingler les actions GitHub par empreinte.** `actions/checkout@v4` suit une étiquette qui peut être déplacée. Piste : remplacer chaque `@v4` par l'empreinte SHA complète du commit correspondant, en gardant la version en commentaire, et laisser Dependabot proposer les mises à jour.
5. **Interdire l'insertion de HTML construit.** Les règles recommandées d'ESLint ne voient pas `.html("<p>" + donnee + "</p>")`. Piste : une règle `no-restricted-syntax` avec un sélecteur qui vise les appels `.html()` ou `.append()` dont l'argument n'est pas une chaîne littérale, puis vérifier qu'elle signale le code naïf de 5.3 et pas le code actuel.
6. **Remplacer la modale par l'élément natif `<dialog>`.** Sa méthode `showModal()` rend déjà l'arrière-plan inerte et gère Échap. Piste : supprimer le code correspondant de `js/app.js` et vérifier que les tests de la modale (focus piégé, réouverture rapide) restent verts sans modification.

8. Glossaire

Terme	Définition
AJAX	Requête HTTP faite en arrière-plan par une page, sans la recharger
Analyse statique	Vérification du code sans l'exécuter (ici ESLint)
Asynchrone	Opération dont le résultat arrive plus tard ; le code continue sans l'attendre
`aria-*`	Attributs qui décrivent rôle et état d'un élément aux technologies d'assistance
Bubbling	Propagation d'un événement de l'élément visé vers tous ses ancêtres
CDN	Réseau de serveurs qui distribue des fichiers publics, ici jQuery
CORS	Règles du navigateur qui limitent les requêtes d'une origine vers une autre
Debounce	Attendre une pause dans une série d'événements avant d'agir
Délégation	Écouteur posé sur un parent stable, valable pour des descendants créés plus tard
Focus	Élément qui reçoit actuellement les frappes du clavier
Formateur	Outil qui réécrit le code dans un style unique (ici Prettier)
Fonction pure	Fonction dont le résultat ne dépend que de ses arguments, sans effet de bord
`inert`	Attribut qui rend un élément et ses descendants non cliquables et non focalisables
Moindre privilège	Ne donner à un programme que les droits strictement nécessaires
NFD	Normalisation Unicode qui sépare une lettre accentuée en lettre de base plus accent
`prefers-reduced-motion`	Requête média : l'utilisateur demande moins d'animations
SRI	Attribut `integrity` : le navigateur vérifie l'empreinte d'un fichier externe
Test de non-régression	Test qui reproduit un bug corrigé, pour qu'il ne revienne pas
XSS	Faille où un contenu injecté dans une page est exécuté comme du code